Can a language model be random enough to be evolution?

Genetic drift with a large language model as the random number source

Sara Behnamian
Department of Biology, Lund University, Sweden
sara.behnamian@biol.lu.se

Abstract

Genetic drift is sampling noise: in the Wright–Fisher model the next generation's allele count is a binomial draw. Large language models (LLMs) are increasingly used inside simulations and agent-based models, and they are known to produce "random" output with human-like biases. We asked what happens to drift when an LLM supplies the randomness, and expressed the answer in population-genetic units, as the variance effective population size N_e implied by the LLM's sampling. Using 1,193 independent calls to three models of one family (Haiku, Sonnet and Opus), we measured raw bit sequences, single-generation Wright–Fisher draws, multi-generation trajectories built from one fresh call per generation or from one call per whole trajectory, and an "RNG mode" in which LLM bits drive an otherwise exact simulation. All three models over-alternated (alternation rate 0.615–0.664 against 0.5) and exaggerated requested probabilities rather than shrinking them toward one half (logit slope 1.22–1.32). When asked for the next generation directly, the models got the mean right but the variance too small: N_e/N was 1.04–1.53 at $N = 200$. In multi-generation runs the excess was larger, up to $N_e/N = 4.59$ (Haiku, one call per generation), so heterozygosity decayed too slowly and alleles fixed too late. Used as a bit source, LLM output was so locally balanced that N_e/N reached 13.9 for $N = 10$, far more than a first-order Markov model of the alternation bias predicts; the full autocorrelation function accounts for it. The direction of selection was respected, and in whole-trajectory runs its effect was exaggerated (effective s 0.064–0.072 for a stated $s = 0.05$). The largest model was the closest to Wright–Fisher behaviour in every direct mode, but its raw bits were no better than those of the smaller models and were the most repetitive across calls. LLM-generated randomness is therefore a poor substitute for a pseudo-random number generator in evolutionary models: it makes drift too weak, and the size of the error depends on model, prompt and how calls are batched.

1 Introduction

Genetic drift is the change in allele frequencies caused by the random sampling of gametes in a finite population [10, 32]. In the Wright–Fisher (WF) model, a diploid population of N individuals carries $2N$ gene copies, and given an allele frequency p the next generation's count of that allele is Binomial$(2N, p)$. The variance of the change, $\mathrm{Var}(\Delta p) = p(1-p)/2N$, sets the pace of everything that follows: heterozygosity decays by a factor $1 - 1/2N$ per generation, a neutral allele fixes with probability equal to its starting frequency, and absorption takes on the order of $4N$ generations [7, 21, 22]. Real populations rarely match the idealised model, so population geneticists describe them by an effective size N_e, the size of an ideal WF population with the same rate of drift [4, 6].

The quality of the randomness matters for simulation. Pseudo-random number generators are tested with large statistical batteries [23], and subtle correlations in a generator that passed common tests have produced wrong answers in Monte Carlo physics [9]. At the same time, LLMs are being placed inside simulations: as agents in social sandboxes [27], as simulated economic actors or survey respondents [1, 2, 19], in agent-based models more broadly [11], and as mutation or crossover operators in evolutionary computation [24, 25]. In such systems an LLM is often the

only source of stochastic behaviour. If it is asked to "simulate" a random process, its sampling errors become part of the model.

It is already well established that LLMs are poor random generators. Asked to flip coins, models reproduce and amplify human biases: too much alternation, too few long runs and too many balanced sequences [30]. Asked for random numbers, they favour a few values [5, 35], avoid repeats and neighbouring numbers [17], and fail to match target distributions in batch and stateless settings [13, 14, 18, 20, 36]. Biased coins come out biased in the wrong way [16, 33], and models do not randomise well in games [15, 34]. These patterns echo the human literature: people over-alternate, expect small samples to be representative, and see streaks where there are none [3, 8, 12, 28, 29, 31], although the strength of the human deficit depends on the task [26].

What has not been done is to follow these biases into a stochastic model whose output depends on them in a known way. Genetic drift is a natural test case because its theory is exact, and because its central quantity, the variance of a binomial draw, is exactly what biased random generation distorts. Our contribution is to express LLM randomness in population-genetic units. We (i) quantify bias, alternation, run lengths and probability distortion in LLM bit sequences, (ii) ask LLMs to carry out WF sampling directly and estimate N_e/N, (iii) run multi-generation trajectories with and without memory and compare heterozygosity decay, fixation probability, absorption time and the response to selection with exact theory and a numpy baseline, (iv) feed LLM bits into an otherwise exact WF simulation ("RNG mode") to separate bad randomness from a poor model of the process, and (v) compare three model sizes. A short theory section links simple parametric models of LLM randomness to N_e, heterozygosity and fixation.

2 Theory

Full derivations are in `theory/derivations.md`, and the script `theory/check_theory.py` checks them numerically. For any process that maps p to $p + \Delta p$, we use the variance effective size

$$N_e = \frac{p(1-p)}{2\,\mathrm{Var}(\Delta p \mid p)}, \tag{1}$$

so that $N_e = N$ for exact WF. $N_e/N > 1$ means drift is too slow.

Bit sequences. We model LLM bits $b_1, b_2, \ldots$ requested with probability p as a stationary two-state Markov chain with produced frequency π and lag-1 autocorrelation ρ ($\rho < 0$: excess alternation). Then the alternation rate is $2\pi(1-\pi)(1-\rho)$, runs of ones are geometric with continuation probability $\pi + \rho(1-\pi)$, and the sum of n bits has variance

$$\mathrm{Var}(S_n) = n\pi(1-\pi)\left[\frac{1+\rho}{1-\rho} - \frac{2\rho(1-\rho^n)}{n(1-\rho)^2}\right]. \tag{2}$$

The distortion of the requested probability is summarised by a logit-linear curve, $\mathrm{logit}\,\pi = \beta_0 + \beta_1 \,\mathrm{logit}\,p$; $\beta_1 < 1$ is shrinkage toward one half and $\beta_1 > 1$ is exaggeration.

RNG mode. If each of the $2N$ offspring copies inherits allele A when its LLM bit is 1, with bits requested at the current p, then $\mathrm{E}[\Delta p] = \pi - p$ and, from (2),

$$\frac{N_e}{N} \approx \frac{p(1-p)}{\pi(1-\pi)} \cdot \frac{1-\rho}{1+\rho}. \tag{3}$$

Excess alternation slows drift ($\rho = -0.2$ gives $N_e/N \approx 1.5$). The first-order chain is only a model; more generally $\mathrm{Var}(S_n) = \sum_{i,j} \gamma_{|i-j|}$ with γ_k the lag-k autocovariance, and we evaluate both forms against data. Exaggeration ($\beta_1 > 1$) adds a systematic $\mathrm{E}[\Delta p]$ that pushes frequencies away from one half, toward the nearest boundary.

Direct mode. When an LLM produces next-generation counts, we describe them by a conditional mean $m(p)$ and a dispersion factor ϕ, with $\mathrm{Var}(X \mid p) = \phi\, 2N\, m(1-m)$. Then $N_e/N = p(1-p)/(\phi\, m(1-m)) \approx 1/\phi$ when $m \approx p$. If the mean is unbiased, heterozygosity $H = 2p(1-p)$ obeys

$$\mathrm{E}[H_t] = H_0 \left(1 - \frac{\phi}{2N}\right)^t = H_0 \left(1 - \frac{1}{2N_e}\right)^t. \tag{4}$$

With drift $M(p) = m(p) - p$ and variance $V(p) = \phi p(1-p)/2N$, the diffusion fixation probability is $u(p_0) = \int_0^{p_0} G / \int_0^1 G$ with $G(x) = \exp(-\int 2M/V)$. Two consequences follow. First, if $M = 0$, then $u(p_0) = p_0$ whatever ϕ is: dispersion errors alone leave neutral fixation probabilities unchanged and only slow the process; the mean time to absorption, $-4N_e[p_0 \ln p_0 + (1-p_0)\ln(1-p_0)]$, scales with N_e [22]. Second, if selection is represented as $M = s_{\mathrm{eff}}\, p(1-p)$, Kimura's result becomes

$$u(p_0) = \frac{1 - e^{-4N_e s_{\mathrm{eff}} p_0}}{1 - e^{-4N_e s_{\mathrm{eff}}}}, \tag{5}$$

so an LLM that gets s right but has $N_e/N > 1$ still exaggerates the effect of selection, because only the product $N_e s_{\mathrm{eff}}$ matters [21]. In whole trajectories with an unbiased conditional mean, successive increments Δp_t are uncorrelated; positive correlation means trajectories that are too smooth and negative correlation means too many reversals.

Checks. Simulated Markov chains matched (2) to within 2.1% and (3) to within 1.5%. Simulations of the dispersion model reproduced (4), neutral fixation at p_0 for $\phi = 0.5$ and $\phi = 2$, and (5); the diffusion results were less accurate for $N = 10$ with strong under-dispersion, where count discreteness matters (details in the theory note).

3 Methods

3.1 LLM calls

No API key was available, so each LLM call was made by a fresh subagent launched by the Claude Code agent harness, using the three model tiers available to subagents (Haiku, Sonnet, Opus). Each subagent received one prompt and no other context from the experiment, ran at low reasoning effort, and had to return its answer through a structured-output tool validated against a JSON schema (a string of bits, a list of integers, or a list of trajectories). Every prompt began with the instruction to produce the random output directly and not to write or run code, use other tools, or use any external source of randomness. We parsed every subagent transcript: in all 1,193 calls the only tool used was the structured-output tool (0 exceptions), and every call returned schema-valid JSON (1,193 of 1,193). There were no refusals. Prompts, raw responses, timestamps and the served model family of every call are stored in `data/raw/calls.jsonl`; the scripts that generated the calls, ingested the transcripts and analysed them are in `src/`. All numpy simulations use fixed seeds. Median wall time per call was 3.3–4.6 s. Because each subagent carries the harness's system prompt, calls consumed about 83 million input tokens in total, almost all of it cached context; output was small.

Replicate calls of the same condition carried a different six-character "run code" (a hash of the call identifier, described as a bookkeeping label), because identical prompts would otherwise invite identical answers. A separate determinism test repeated two prompts four times each without a run code.

3.2 Design

A pilot of 30 calls checked JSON validity, refusals, timing and output shape across all modes and models. The full run had 1,163 calls:

- **Bit sequences** (264 calls): 300 bits per call, requested $p \in \{0.05, 0.2, 0.5, 0.8\}$, two phrasings (plain; "you are a random number generator"), 8 replicates per cell and 20 at $p = 0.5$, whose bits also fed RNG mode. We analysed the first 300 digits of each answer and report length errors separately.
- **Direct mode, one generation** (324 calls): 20 next-generation draws per call from a stated state, explicitly required to be independent. Full grid $N \in \{10, 50, 200\}$, $p \in \{0.05, 0.2, 0.5, 0.8\}$; neutral cells with two phrasings (plain; "simulate exactly as a Wright–Fisher process") and three calls each, selection cells ($s = 0.05$, WF phrasing) with two calls each, and the third phrasing (random number generator) at $N = 50$ only.
- **Fresh call per generation** (464 calls): each generation of a set of populations was one new call that saw only the current counts and the generation number. To keep the call count down, up to 40 independent populations were batched in one call. $N = 10$ with 20 populations at each $p_0 \in \{0.05, 0.2, 0.5, 0.8\}$ for 40 generations, with $s = 0$ and $s = 0.05$; and $N = 50$, 40 populations at $p_0 = 0.5$, $s = 0$, for 30 generations. Absorbed populations were dropped from later calls.
- **Whole trajectory per call** (87 calls): four independent trajectories per call. $N = 10$, all four p_0, $s \in \{0, 0.05\}$, 40 generations, 3 calls per cell; $N = 50$, $p_0 = 0.5$, 30 generations, 5 calls.
- **Determinism test** (12+12 calls).

The multi-generation part of the requested grid was trimmed to keep the total near 1,200 calls: trajectories were run only for $N = 10$ (all p_0, both s) and $N = 50$ ($p_0 = 0.5$, $s = 0$), and the selection and third-phrasing cells of direct mode have fewer calls.

3.3 Analysis

For direct mode we pooled draws within each cell and estimated $\phi = \widehat{\mathrm{Var}}(X/2N)/[m(1-m)/2N]$, with m the WF expectation after selection; $N_e/N = 1/\phi$. Summaries per model and N average ϕ over neutral cells (plain and WF phrasings) and invert it. Confidence intervals come from bootstrapping calls within cells (2,000 resamples for cells, 500 for summaries), which keeps the within-call dependence intact. For trajectories we estimated N_e from all transitions starting at polymorphic states, $\hat{N}_e = \sum p_t(1-p_t)/2\sum(\Delta p_t)^2$, and an effective selection coefficient by regressing Δp on $p(1-p)$ through the origin. For comparison we ran 5,000 numpy WF replicates of every trajectory design, truncated at the same generation, and we report fixation as "fixed by the last generation" for LLM and numpy runs alike. In RNG mode, the bits of each model and requested p were concatenated in call order and cut into non-overlapping windows of $2N$ bits, each window giving one offspring generation. For a multi-generation version we turned the $p = 0.5$ bits into Bernoulli(p) draws with the standard comparison method (two bits per draw on average) and ran $N = 10$ populations from $p_0 = 0.5$ until the bits ran out; because that gives only a handful of trajectories, we also ran 3,000 trajectories driven by a Markov chain fitted to the same bits.

4 Results

4.1 Raw randomness

At $p = 0.5$ all three models produced balanced frequencies but alternated far too often (Table 1): the alternation rate was 0.615 (Opus), 0.638 (Haiku) and 0.664 (Sonnet), against 0.5, and the lag-1 autocorrelation was -0.23 to -0.33. Long runs were missing: the longest run of identical digits in any 300-bit answer was 3 for Sonnet and 4 for Opus, where a random sequence of

Table 1: Bit sequences requested with $p = 0.5$ (pooled phrasings, 40 calls and about 12,000 bits per model), and the slope of the logit shrinkage curve fitted across $p \in \{0.05, 0.2, 0.5, 0.8\}$. The iid row gives the expectation for independent fair bits; the longest run is the median over simulated 300-bit sequences.

Model	freq. of 1s	alternation	$\hat{\rho}_1$	mean run	longest run	entropy rate	logit slope β_1 [95% CI]
iid ideal	0.500	0.500	0	2.00	8 (median)	1.000	1
Haiku	0.511	0.638	-0.28	1.60	7	0.886	1.32 [1.22, 1.45]
Sonnet	0.509	0.664	-0.33	1.53	3	0.825	1.23 [1.21, 1.25]
Opus	0.514	0.615	-0.23	1.67	4	0.934	1.22 [1.20, 1.24]

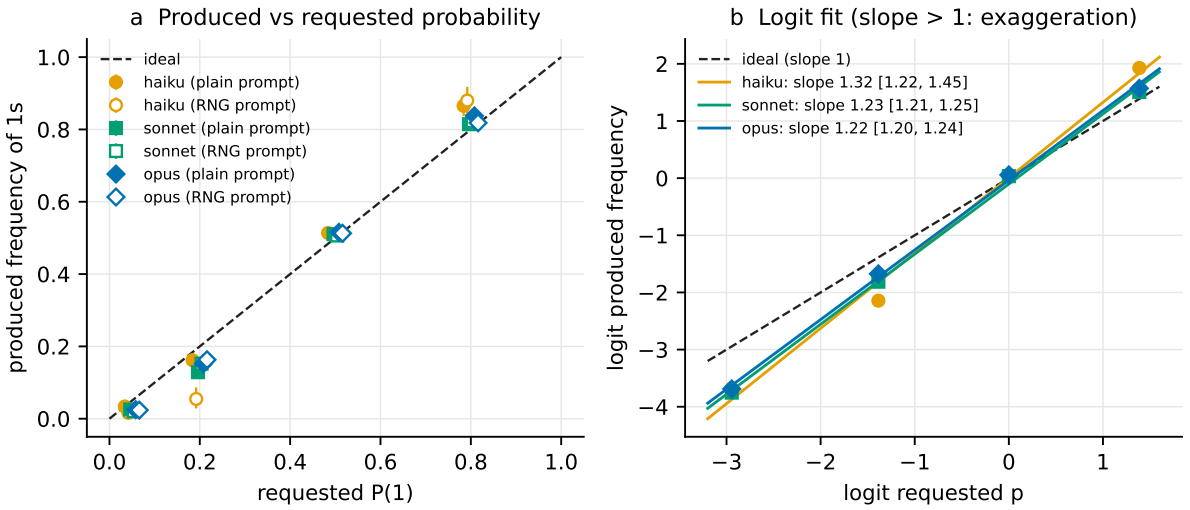


Figure 1: Produced versus requested probability in LLM bit sequences. (a) Pooled frequency of 1s per model and phrasing with 95% bootstrap intervals over calls. (b) Logit-linear fit; slopes above 1 mean the requested probability is exaggerated, so rare outcomes are under-produced.

that length typically contains a run of 8 (Fig. 2). The dependence was not first-order. The autocorrelation alternated in sign with lag, with a strong negative value at lag 4 (-0.20, -0.27, -0.27 for Haiku, Sonnet, Opus) and positive values at lag 3 (Fig. 8a), the signature of short repeating motifs. Block-entropy estimates of the entropy rate were 0.825–0.934 bits per digit.

For other probabilities, the models did not shrink toward one half, as a "hedging" model would predict. They exaggerated (Fig. 1). At $p = 0.05$ the produced frequency was 0.023–0.025, at $p = 0.2$ it was 0.105–0.158, and at $p = 0.8$ it was 0.819–0.873. The fitted logit slopes were 1.22 [1.20, 1.24] for Opus, 1.23 [1.21, 1.25] for Sonnet and 1.32 [1.22, 1.45] for Haiku. Rare outcomes were also spaced too evenly: at $p = 0.2$ runs of zeros peaked at four to six instead of decaying geometrically from one (Fig. 2b), and ones almost never appeared in pairs.

Two practical problems are worth noting. Output length was rarely right: none of Haiku's answers had exactly 300 digits (range 174–3816, with some answers degenerating into long blocks of zeros), against 33% for Opus. And calls were not independent of each other even with distinct run codes: pairs of Opus answers at $p = 0.5$ shared their first 21.7 digits on average, and up to 13 of 20 Opus answers began with the same 16 digits (Sonnet 9.3, Haiku 4.2). Repeating a prompt verbatim gave no exact duplicates, but Opus answers again shared long prefixes (18.2 digits on average).

4.2 The LLM as the Wright–Fisher process

Asked for the next generation directly, every model returned valid integers in range (0 invalid values among 6,720 draws) and centred them correctly: the median absolute difference between the mean draw and the WF expectation was 0.0013 in frequency units across neutral cells

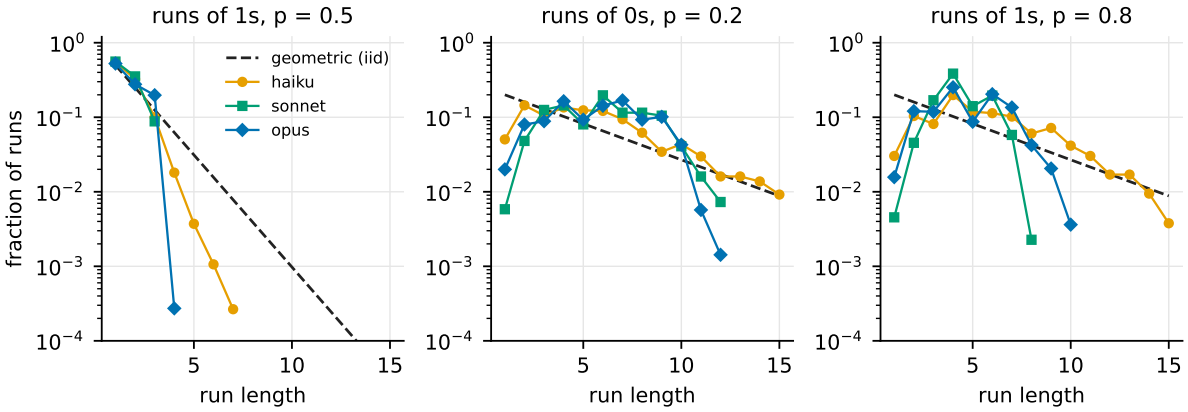


Figure 2: Run-length distributions of LLM bits against the geometric distribution expected for independent bits at the requested probability. Runs are counted within calls.

Table 2: Effective population size relative to the stated size, N_e/N, by model and mode. Direct mode: neutral cells averaged over p and two phrasings. Trajectory modes: $p_0 = 0.5$, $s = 0$, estimated from all polymorphic transitions. RNG mode: one LLM bit per offspring copy, bits requested at $p = 0.5$. Exact WF gives 1.

	direct, one generation			fresh call per generation		whole trajectory		RNG mode
Model	$N = 10$	$N = 50$	$N = 200$	$N = 10$	$N = 50$	$N = 10$	$N = 50$	$N = 10$, $p = 0.5$
Haiku	1.21	1.52	1.53	4.59	3.46	2.18	2.01	6.5
Sonnet	1.09	1.46	1.52	3.56	2.97	1.44	1.80	13.9
Opus	1.18	1.06	1.04	1.65	1.28	1.16	1.31	12.7

(largest 0.042). The variance was too small (Fig. 3a). N_e/N was 1.21, 1.52 and 1.53 for Haiku at $N = 10, 50, 200$, 1.09, 1.46 and 1.52 for Sonnet, and 1.18, 1.06 and 1.04 for Opus (Table 2; Fig. 3b). Only Opus at $N \geq 50$ was close to 1, and even there the intervals (1.01–1.08 at $N = 200$) excluded it. The phrasing made little difference: at $N = 50$, Haiku gave 1.59 (plain), 1.46 (WF) and 1.52 (random number generator); Sonnet 1.40, 1.53 and 1.19; Opus 1.11, 1.01 and 1.08. There was no preference for multiples of five (0.21 of draws at $N \geq 50$, against 0.20 under the binomial) and only a slight excess of the exact expected count (0.12–0.13 against 0.09).

Batching created dependence of two kinds (Fig. 8b). Within a call, successive draws were negatively correlated (mean lag-1 correlation -0.44 for Haiku, -0.37 for Sonnet, -0.33 for Opus; zero for independent draws), the count-level version of over-alternation. Between calls, the first draw was almost the same in every replicate call of a cell: its variance across calls was only 0.09–0.10 of the binomial variance, against 0.70–0.96 for later positions, and in 0.36 of replicate pairs the first three draws were identical. The mean of each batch of 20 was also too accurate. For independent draws the variance of a 20-draw mean is one twentieth of the single-draw variance, or 0.05 in our units; the observed values were 0.001 (Opus), 0.003 (Sonnet) and 0.017 (Haiku). Each batch was built to look like a correct sample, which is the law of small numbers in its plainest form [29].

4.3 Consequences over many generations

In multi-generation runs the shortfall in variance grew (Fig. 4). With a fresh call per generation, N_e/N at $N = 10$ was 4.59 for Haiku, 3.56 for Sonnet and 1.65 for Opus (at $N = 50$: 3.46, 2.97, 1.28). Heterozygosity decayed accordingly (Fig. 5): after 40 generations at $N = 10$ it was 0.478 for Haiku, 0.189 for Sonnet and 0.093 for Opus, against 0.065 in numpy WF and in theory. Haiku's populations barely drifted at all. None of its neutral populations starting at 0.5 or 0.8

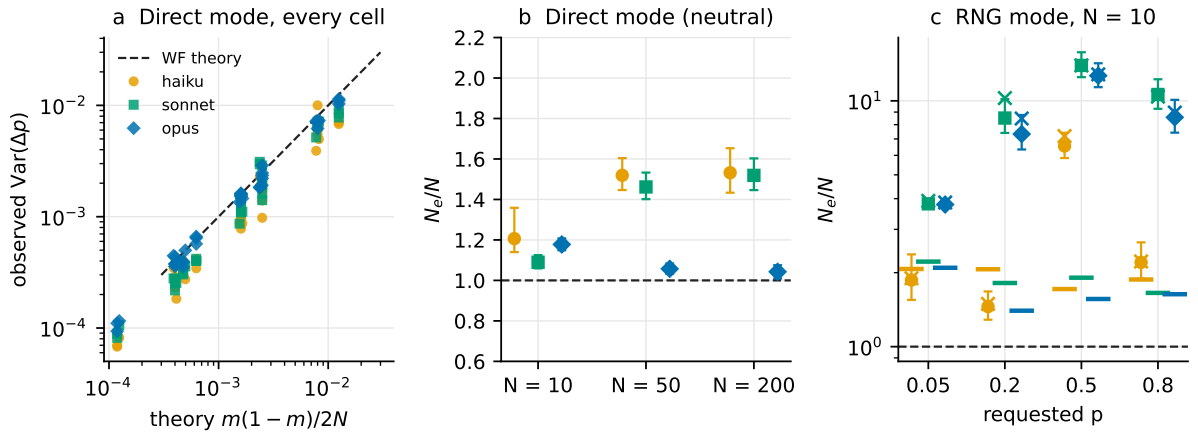


Figure 3: Variance of one-generation change. (a) Observed Var(Δp) in every direct-mode cell against the WF expectation (dashed line). (b) N_e/N in direct mode per model and N, with 95% intervals from a call-level bootstrap. (c) RNG mode, $N = 10$: N_e/N when each offspring copy uses one LLM bit requested at the current p; bars show the prediction of the first-order Markov model (Eq. 3) and crosses the prediction from the full empirical autocorrelation function. Note the log scale in (c).

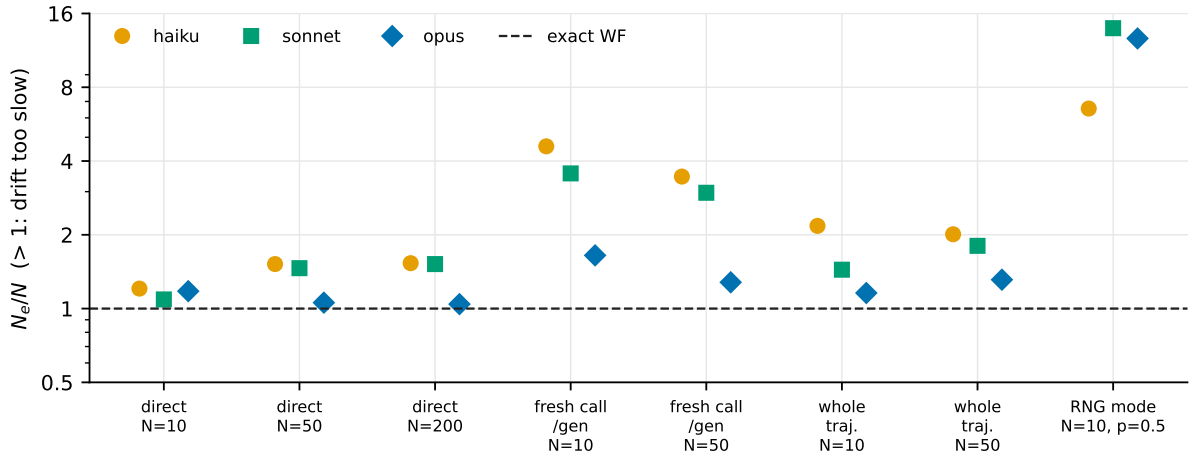


Figure 4: N_e/N for each model across all modes (values in Table 2). The dashed line is exact Wright–Fisher drift.

fixed within 40 generations, while populations starting from a single copy were lost within a few generations; in numpy WF, 0.41 of populations starting at 0.5 had fixed by then. Sonnet fixed 0.15 and Opus 0.40 (Fig. 6a). These differences come mainly from variance, not from a biased mean. Regressing Δp on $p - 0.5$ gave slopes of 0.005 (Opus), 0.019 (Sonnet) and -0.027 (Haiku) at $N = 10$, a weak pull toward the middle for Haiku only, whereas the per-transition dispersion factor in the middle frequency range was 0.19 (Haiku), 0.26 (Sonnet) and 0.64 (Opus). Fresh-call runs batched 40 populations per call, so they are not memoryless across populations, and their dispersion was lower than in direct mode, where one state was replicated 20 times.

When one call produced a whole trajectory, N_e/N was 2.18 (Haiku), 1.44 (Sonnet) and 1.16 (Opus) at $N = 10$. These trajectories looked plausible at a glance (Fig. 7), and every one of them stayed absorbed once it reached 0 or $2N$; 7 of the 344 trajectories had the wrong length. Their texture was wrong in a model-specific way. Sonnet and Opus produced too many reversals: the lag-1 correlation of successive increments was -0.17 and -0.29, and the share of consecutive non-zero steps that changed direction was 0.61 and 0.68, against 0.49 in numpy. Haiku's trajectories were smoother instead (lag-1 correlation 0.10, reversal rate 0.41), with long

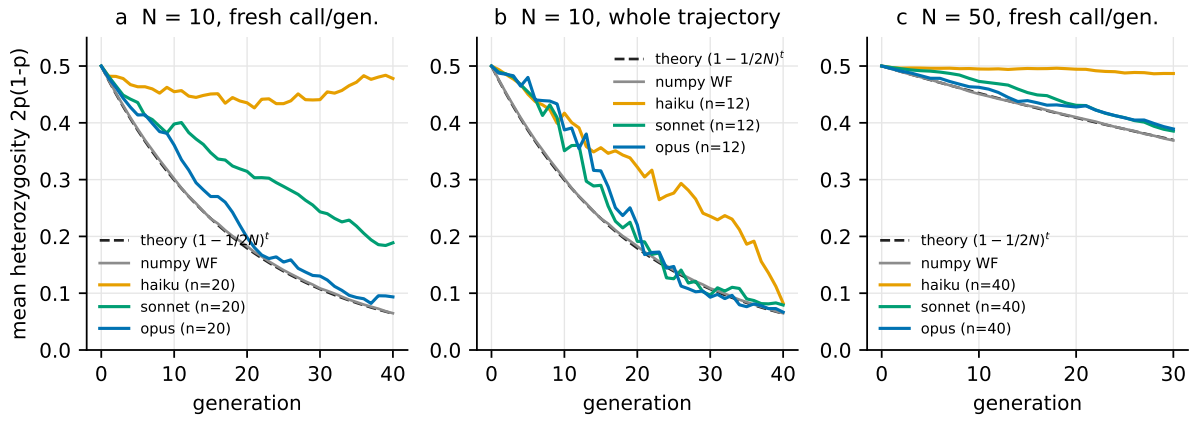


Figure 5: Mean heterozygosity $2p(1-p)$ over generations for populations starting at $p_0 = 0.5$, $s = 0$, against theory (Eq. 4 with $N_e = N$) and 5,000 numpy WF replicates. n is the number of LLM populations.

Table 3: Effective selection coefficient for a stated $s = 0.05$. Direct mode: mean over cells of $(\bar{p}' - p)/[p(1-\bar{p}')]$. Trajectory modes: regression of Δp on $p(1-p)$ over all polymorphic transitions; the WF expectation at the same states is 0.049.

	direct, one generation			fresh call/gen.	whole trajectory
Model	$N = 10$	$N = 50$	$N = 200$	$N = 10$	$N = 10$
Haiku	0.016	0.032	0.043	0.045	0.072
Sonnet	0.104	0.076	0.054	0.065	0.064
Opus	0.046	0.041	0.051	0.052	0.069

monotone climbs and descents (Fig. 7). Whole trajectories also flattened the relation between starting frequency and fixation (Fig. 6a): a single initial copy ($p_0 = 0.05$) went on to fix in 0.25 (Opus) to 0.33 (Haiku and Sonnet) of trajectories by generation 40, against 0.03 in numpy WF and an eventual probability of 0.05, while Haiku fixed only 0.25 of trajectories that started at 0.8 (numpy 0.74). These cells hold only 12 trajectories each, but the pattern suggests that in a single call the models tell stories in which rare alleles rise more often than chance allows. Fresh-call trajectories had increment correlations close to zero (-0.03 to 0.06 at $N = 10$), as expected when each step is a separate call (Fig. 8c).

4.4 Selection

All models moved frequencies in the favoured direction (Table 3). In fresh-call runs the effective selection coefficient estimated from all transitions was 0.045 (Haiku), 0.052 (Opus) and 0.065 (Sonnet), against 0.049 for the WF expectation at the same states. In whole-trajectory runs it was higher for every model, 0.064–0.072, an exaggeration of 32–47%. In single-generation direct mode the estimates rest on two calls per cell and were noisy; Haiku under-responded at $N = 10$ (0.016) and Sonnet over-responded (0.104). Because $N_e/N > 1$ as well, the product $N_e s$ that governs fixation was inflated even where s_{eff} was right. Fixation by generation 40 with $s = 0.05$ (Fig. 6b) nonetheless stayed close to the numpy values for Sonnet and Opus, possibly because slow drift also delays fixation within a fixed time window, so that the two errors partly cancel. Haiku again fixed rarely (0.20 from $p_0 = 0.5$ against 0.62 in numpy).

4.5 RNG mode: bad randomness without a model of the process

When LLM bits drove an otherwise exact WF step, drift was much too slow (Fig. 3c). With one bit per offspring copy and $N = 10$, N_e/N at $p = 0.5$ was 6.5 (Haiku), 13.9 (Sonnet) and 12.7

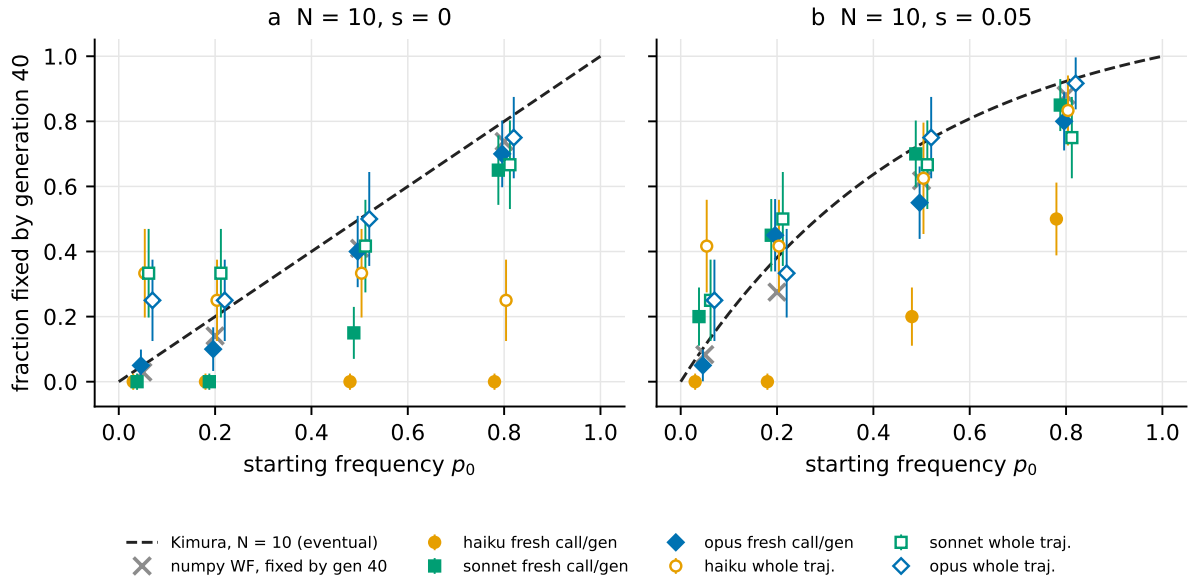


Figure 6: Fraction of $N = 10$ populations fixed by generation 40 against starting frequency, without (a) and with (b) selection. Filled symbols: fresh call per generation (20 populations per point); open symbols: whole trajectory per call (12 per point). Error bars are binomial standard errors. Grey crosses: numpy WF truncated at the same generation. Dashed lines: eventual fixation probability (p_0, or Kimura's formula with $N_e = N$).

(Opus), and at $N = 50$ it was 9.2, 28.6 and 33.2. The first-order Markov model of Eq. (3), with the fitted ρ, predicted only 1.6–1.9. A prediction from the full empirical autocorrelation function, which includes the negative correlations at lags 2 and 4 and beyond, matched the observed values at $N = 10$ to within 21% (7.2, 13.9, 12.8). The LLM bits are balanced over windows of tens of digits, so the sum of 20 bits varies far less than a binomial count. At $p = 0.2$ and $p = 0.05$ the excess was smaller but still present for $N = 10$ (1.5–8.5 at $p = 0.2$). For larger windows at $p \neq 0.5$ the estimate fell below 1 for some models, because windows then spanned several calls whose overall frequencies differed; the between-call heterogeneity adds variance that the within-call balancing removes. Exaggeration of p also gave a systematic $\mathrm{E}[\Delta p] = \pi - p$ that pushed rare alleles further down and common ones further up, which in a multi-generation run acts like positive frequency dependence.

Converting fair LLM bits into Bernoulli(p) draws with the comparison method gave only 8–12 complete $N = 10$ trajectories per model, so these numbers are rough. None of them fixed (all were lost), and their N_e/N was 3.3–3.7. A Markov chain fitted to the same bits, run for 3,000 trajectories, fixed in only 0.15–0.24 of cases instead of 0.5. The comparison method maps alternation bias into a probability error that depends on the binary expansion of p, so a correlated bit source does not just add noise but introduces a directional force.

4.6 Model comparison

A larger model was not uniformly a better random source. Opus was the closest to WF behaviour in every mode where the model simulates the process (Table 2; direct N_e/N 1.06–1.18, fresh-call 1.28–1.65, whole trajectory 1.16–1.31), and Haiku was the furthest. Sonnet sat in between in the trajectory modes but matched Haiku in direct mode at $N \geq 50$. The ranking does not hold for raw bits. Opus bits alternated slightly less than the others, yet in RNG mode they slowed drift as much as Sonnet's (12.7 against 13.9, Haiku 6.5), and Opus was the most repetitive across calls. All three models exaggerated probabilities to a similar degree.

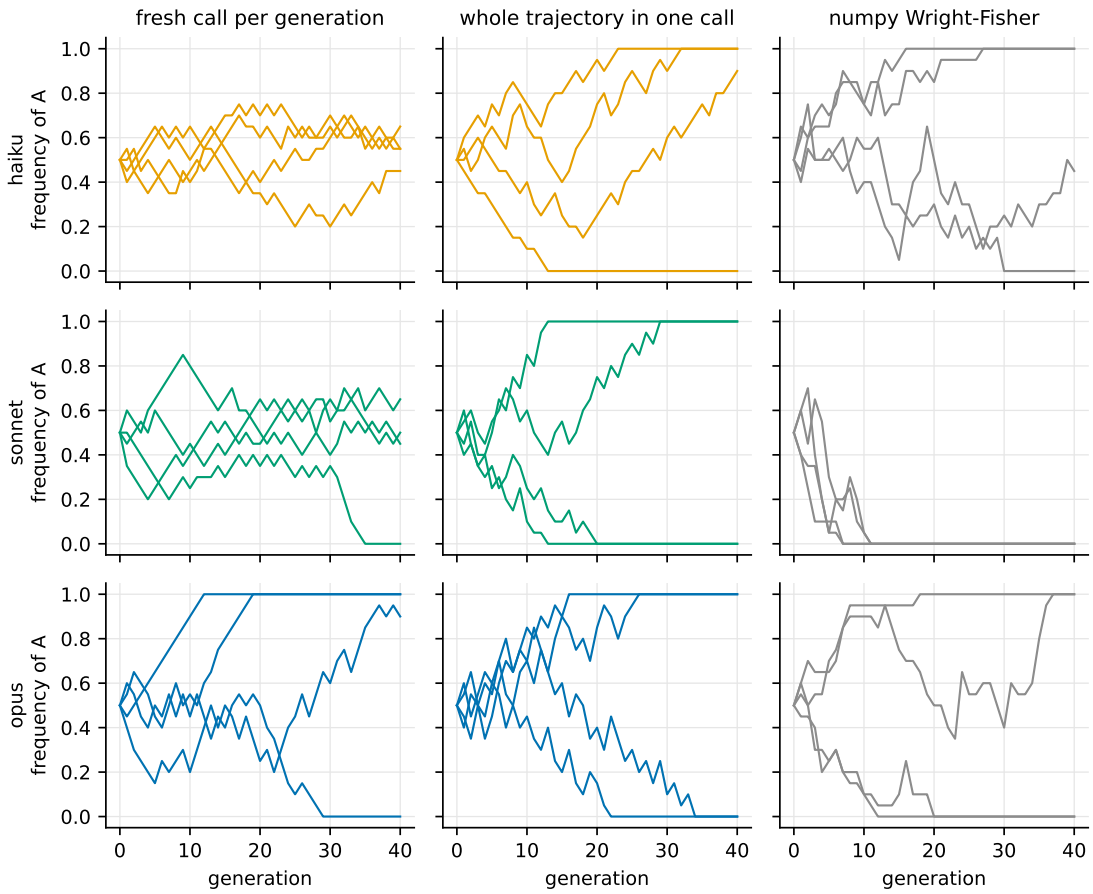


Figure 7: Example trajectories at $N = 10$, $p_0 = 0.5$, $s = 0$: four populations from fresh-call runs, four from one whole-trajectory call, and four numpy WF replicates per row.

5 Discussion

The answer to the question in the title is no, at least for the models we tested. When an LLM is the random number source, drift runs too slowly. In population-genetic terms the LLM behaves like a population that is larger than stated, 1.04 to 1.53 times larger when a single state is resampled, 1.2 to 4.6 times larger in multi-generation simulations, and 6.5 to 13.9 times larger when its bits are used directly as a random source.

The error has the same root as the human biases described for random sequence generation [8, 31]. Over-alternation and the avoidance of long runs make sums of LLM bits less variable than binomial sums. When the model produces counts directly, it builds batches of draws that look like a correct sample, spread around the right mean but too tightly and too evenly. Earlier work found the same over-alternation in LLM coin flips [30] and too little spread across simulated individuals [1]; what we add is that these errors translate into a quantifiable change in N_e, and through it into slower loss of heterozygosity, delayed fixation and a stronger effect of selection.

Two results were not what a simple model predicted. First, the models did not hedge toward one half when asked for a biased coin; they made rare events rarer. This differs from a probability-weighting view in which small probabilities are overweighted, and it matters for population genetics because a source that under-produces rare outcomes pushes rare alleles toward loss. Second, a first-order Markov model with the measured alternation rate underestimated the effect of the bits on drift by a factor of up to 8. LLM bit strings are balanced over long windows, not just from one digit to the next, so only a model that uses the full autocorrelation function accounts for the variance of their sums. This is a warning against summarising LLM randomness

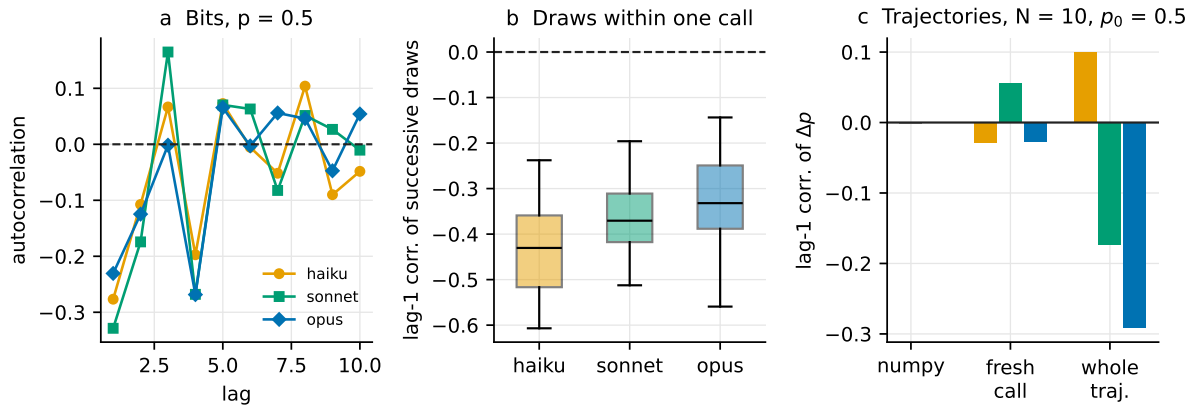


Figure 8: Dependence. (a) Autocorrelation of bits at $p = 0.5$ by lag (pooled within calls). (b) Lag-1 correlation of successive draws within direct-mode calls, one value per cell. (c) Lag-1 correlation of successive increments Δp in trajectories at $N = 10$, $p_0 = 0.5$; numpy WF gives zero.

by a single alternation statistic.

The comparison between modes separates two failures. In RNG mode the model does not know about the population at all, yet drift is still distorted, so poor randomness on its own is enough to bias the outcome. In direct mode the model knows the process, uses the right mean and the right direction of selection, and errs mainly in the variance. Its errors are smaller than in RNG mode because it is not composing outcomes digit by digit; it behaves as if it knew roughly how much a binomial count should vary and then compressed that spread. The memory experiment did not support the idea that LLMs imagine drift as smooth. Sonnet and Opus trajectories reversed direction too often, and only Haiku produced overly smooth paths. When each generation was a fresh call, the increments were uncorrelated but too small, so memory changes the texture of trajectories more than their overall rate.

For applied work the conclusions are practical. LLM agents in evolutionary or ecological simulations should draw their random numbers from a proper pseudo-random generator, passed in through the prompt or a tool, as recent work also suggests [14]. If an LLM must generate stochastic outcomes, its effective N_e should be measured with checks like ours before the results are interpreted. Batching, phrasing and model choice all change the size of the error, and identical or near-identical prompts produce correlated answers across calls, so replicate simulations are not independent replicates.

6 Limitations

Every LLM call ran as a subagent inside an agent harness, with a large system prompt and tools available that the model was told not to use. We could not set the sampling temperature, which is known to affect LLM coin flips [30], and all calls used low reasoning effort. We tested three models of one family at one point in time, so the numbers will not transfer directly to other models or versions. The multi-generation part of the grid was trimmed to $N = 10$ and $N = 50$, the number of populations per cell was small (12 to 40), and fixation was measured at a fixed horizon rather than at absorption. The fresh-call runs batched up to 40 populations in one call and the direct-mode calls asked for 20 draws at once, so they test batched randomness; the first-draw analysis shows that batching itself creates dependence, but a design with one draw per call would cost twenty times more calls. Run codes made replicate prompts differ, but they may themselves act as seeds and their effect was not varied systematically. In RNG mode the windows of bits for large N crossed call boundaries, which mixes within-call balance with between-call heterogeneity, and the multi-generation fair-coin runs contain only 8 to 12 trajectories per model.

Finally, our parametric models are descriptive: the dispersion factor ϕ summarises the output but does not explain how the model generates it.

Data and code availability

All prompts, raw responses, scripts, results and figures are in the project repository (`https://github.com/sarabehnamian/LLM-random-drift`). `data/raw/calls.jsonl` holds one record per LLM call with its prompt, response, timing and model family; `data/raw/memoryless/` holds the fresh-call trajectories; `src/design.py` generates the calls, `src/ingest.py` extracts them from the agent transcripts, `src/analysis.py`, `src/figures.py` and `src/summary.py` produce every number, figure and table in this paper, and `theory/` contains the derivations and their numerical checks.

References

[1] Gati V. Aher, Rosa I. Arriaga, and Adam Tauman Kalai. Using large language models to simulate multiple humans and replicate human subject studies. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 337–371, 2023. URL `https://proceedings.mlr.press/v202/aher23a.html`.

[2] Lisa P. Argyle, Ethan C. Busby, Nancy Fulda, Joshua R. Gubler, Christopher Rytting, and David Wingate. Out of one, many: Using language models to simulate human samples. *Political Analysis*, 31(3):337–351, 2023. doi: 10.1017/pan.2023.2.

[3] Maya Bar-Hillel and Willem A. Wagenaar. The perception of randomness. *Advances in Applied Mathematics*, 12(4):428–454, 1991. doi: 10.1016/0196-8858(91)90029-I.

[4] Brian Charlesworth. Fundamental concepts in genetics: Effective population size and patterns of molecular evolution and variation. *Nature Reviews Genetics*, 10(3):195–205, 2009. doi: 10.1038/nrg2526.

[5] Javier Coronado-Blázquez. Deterministic or probabilistic? The psychology of LLMs as random number generators. arXiv preprint arXiv:2502.19965, 2025.

[6] James F. Crow and Motoo Kimura. *An Introduction to Population Genetics Theory*. Harper & Row, New York, 1970.

[7] Warren J. Ewens. *Mathematical Population Genetics 1: Theoretical Introduction*, volume 27 of *Interdisciplinary Applied Mathematics*. Springer, New York, 2 edition, 2004. doi: 10.1007/978-0-387-21822-9.

[8] Ruma Falk and Clifford Konold. Making sense of randomness: Implicit encoding as a basis for judgment. *Psychological Review*, 104(2):301–318, 1997. doi: 10.1037/0033-295X.104.2.301.

[9] Alan M. Ferrenberg, D. P. Landau, and Y. Joanna Wong. Monte Carlo simulations: Hidden errors from "good" random number generators. *Physical Review Letters*, 69:3382–3384, 1992. doi: 10.1103/PhysRevLett.69.3382.

[10] Ronald A. Fisher. *The Genetical Theory of Natural Selection*. Clarendon Press, Oxford, 1930.

[11] Chen Gao, Xiaochong Lan, Nian Li, Yuan Yuan, Jingtao Ding, Zhilun Zhou, Fengli Xu, and Yong Li. Large language models empowered agent-based modeling and simulation: A

survey and perspectives. *Humanities and Social Sciences Communications*, 11(1), 2024. doi: 10.1057/s41599-024-03611-3.

[12] Thomas Gilovich, Robert Vallone, and Amos Tversky. The hot hand in basketball: On the misperception of random sequences. *Cognitive Psychology*, 17(3):295–314, 1985. doi: 10.1016/0010-0285(85)90010-6.

[13] Jia Gu, Liang Pang, Huawei Shen, and Xueqi Cheng. Do LLMs play dice? exploring probability distribution sampling in large language models for behavioral simulation. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)*, pages 5375–5390, Abu Dhabi, UAE, 2025. URL `https://aclanthology.org/2025.coling-main.360`. arXiv:2404.09043.

[14] Xiangming Gu, Soham De, Michalis Titsias, Larisa Markeeva, Petar Veličković, and Razvan Pascanu. The illusion of stochasticity in LLMs. arXiv preprint arXiv:2604.06543, 2026.

[15] Zihao Guo, Hongtao Lv, Chaoli Zhang, Yibowen Zhao, Yixin Zhang, and Lizhen Cui. The illusion of randomness: How LLMs fail to emulate stochastic decision-making in rock-paper-scissors games? In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 8618–8637, Suzhou, China, 2025. doi: 10.18653/v1/2025.findings-emnlp.458.

[16] Ritwik Gupta, Rodolfo Corona, Jiaxin Ge, Eric Wang, Dan Klein, Trevor Darrell, and David M. Chan. Enough coin flips can make LLMs act Bayesian. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7634–7655, Vienna, Austria, 2025. doi: 10.18653/v1/2025.acl-long.377.

[17] Rachel M. Harrison. A comparison of large language model and human performance on random number generation tasks. arXiv preprint arXiv:2408.09656, 2024.

[18] Aspen K. Hopkins, Alex Renda, and Michael Carbin. Can LLMs generate random numbers? evaluating LLM sampling in controlled domains. In *ICML 2023 Workshop on Sampling and Optimization in Discrete Space (SODS)*, 2023. URL `https://people.csail.mit.edu/renda/llm-sampling-paper`.

[19] John J. Horton. Large language models as simulated economic agents: What can we learn from homo silicus? NBER Working Paper 31122, National Bureau of Economic Research, 2023. URL `https://www.nber.org/papers/w31122`.

[20] Rabimba Karanjai, Yang Lu, Ranjith Chodavarapu, Lei Xu, and Weidong Shi. Evaluating the quality of randomness and entropy in tasks supported by large language models. arXiv preprint arXiv:2510.12080, 2025.

[21] Motoo Kimura. On the probability of fixation of mutant genes in a population. *Genetics*, 47(6):713–719, 1962. doi: 10.1093/genetics/47.6.713.

[22] Motoo Kimura and Tomoko Ohta. The average number of generations until fixation of a mutant gene in a finite population. *Genetics*, 61(3):763–771, 1969. doi: 10.1093/genetics/61.3.763.

[23] Pierre L'Ecuyer and Richard Simard. TestU01: A C library for empirical testing of random number generators. *ACM Transactions on Mathematical Software*, 33, 2007. doi: 10.1145/1268776.1268777.

[24] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. Evolution through large models. arXiv preprint arXiv:2206.08896, 2022.

[25] Elliot Meyerson, Mark J. Nelson, Herbie Bradley, Adam Gaier, Arash Moradi, Amy K. Hoover, and Joel Lehman. Language model crossover: Variation through few-shot prompting. *ACM Transactions on Evolutionary Learning and Optimization*, 4(4):Article 27, 2024. doi: 10.1145/3694791.

[26] Raymond S. Nickerson. The production and perception of randomness. *Psychological Review*, 109(2):330–357, 2002. doi: 10.1037/0033-295X.109.2.330.

[27] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, San Francisco, CA, USA, 2023. ACM. doi: 10.1145/3586183.3606763.

[28] Amnon Rapoport and David V. Budescu. Generation of random series in two-person strictly competitive games. *Journal of Experimental Psychology: General*, 121(3):352–363, 1992. doi: 10.1037/0096-3445.121.3.352.

[29] Amos Tversky and Daniel Kahneman. Belief in the law of small numbers. *Psychological Bulletin*, 76(2):105–110, 1971. doi: 10.1037/h0031322.

[30] Katherine Van Koevering and Jon Kleinberg. How random is random? evaluating the randomness and humaneness of LLMs' coin flips. arXiv preprint arXiv:2406.00092, 2024. Later arXiv revision retitled "Heads, Tails, and AI Fails: LLMs, Randomness, and Human Judgments".

[31] Willem A. Wagenaar. Generation of random sequences by human subjects: A critical survey of literature. *Psychological Bulletin*, 77(1):65–72, 1972. doi: 10.1037/h0032060.

[32] Sewall Wright. Evolution in Mendelian populations. *Genetics*, 16(2):97–159, 1931. doi: 10.1093/genetics/16.2.97.

[33] Tim Z. Xiao, Johannes Zenn, Zhen Liu, Weiyang Liu, Robert Bamler, and Bernhard Schölkopf. Flipping against all odds: Reducing LLM coin flip bias via verbalized rejection sampling. arXiv preprint arXiv:2506.09998, 2025.

[34] Lingyu Yang. Do LLMs know when to flip a coin? strategic randomization through reasoning and experience. arXiv preprint arXiv:2506.18928, 2025.

[35] Yiming Zhang, Avi Schwarzschild, Nicholas Carlini, Zico Kolter, and Daphne Ippolito. Forcing diffuse distributions out of language models. arXiv preprint arXiv:2404.10859, 2024.

[36] Minda Zhao, Yilun Du, and Mengyu Wang. Large language models are bad dice players: LLMs struggle to generate random numbers from statistical distributions. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 22942–22959, San Diego, USA, 2026. doi: 10.18653/v1/2026.acl-long.1051. arXiv:2601.05414.